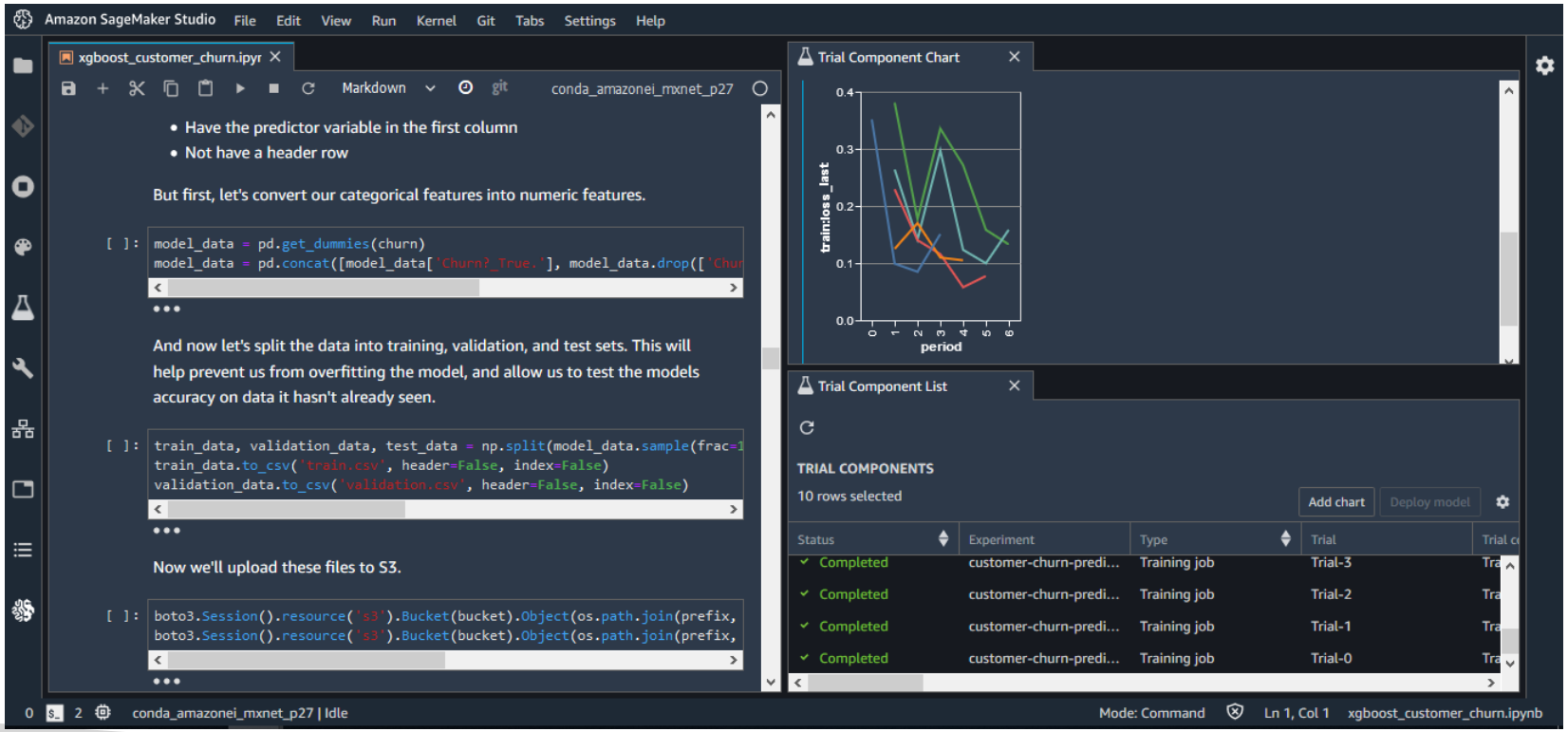
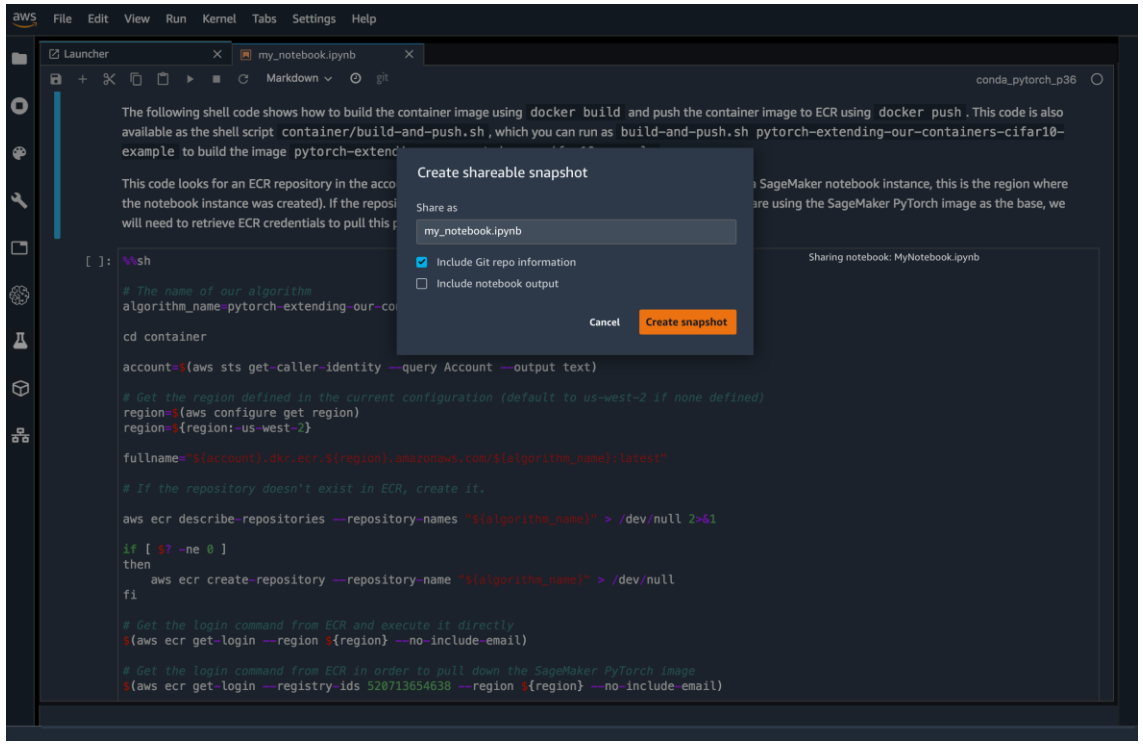
SageMaker Studio

- Visual IDE for machine learning!
- Integrates many of the features we're about to cover.



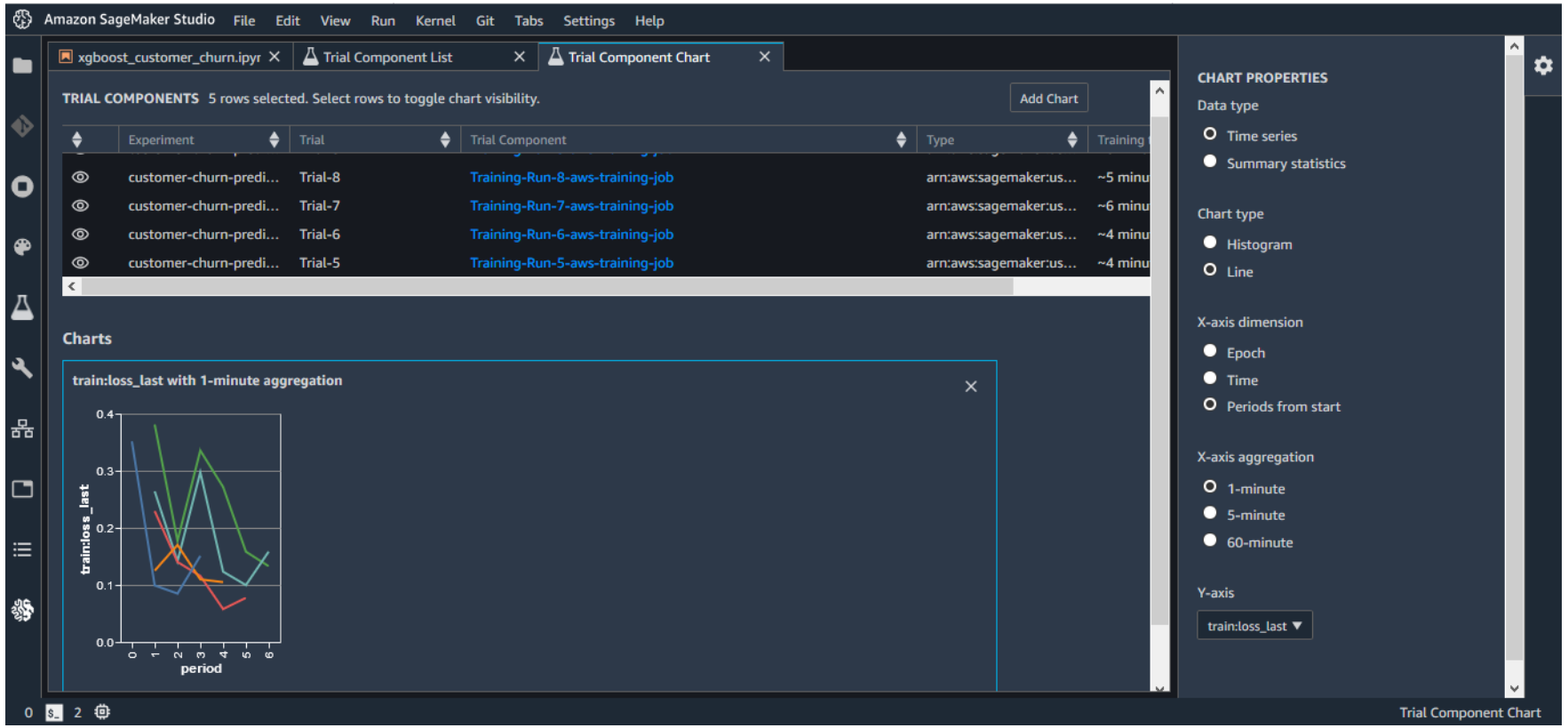
SageMaker Notebooks

- Create and share Jupyter notebooks with SageMaker Studio
- Switch between hardware configurations (no infrastructure to manage)



SageMaker Experiments

- Organize, capture, compare, and search your ML jobs



SageMaker Debugger

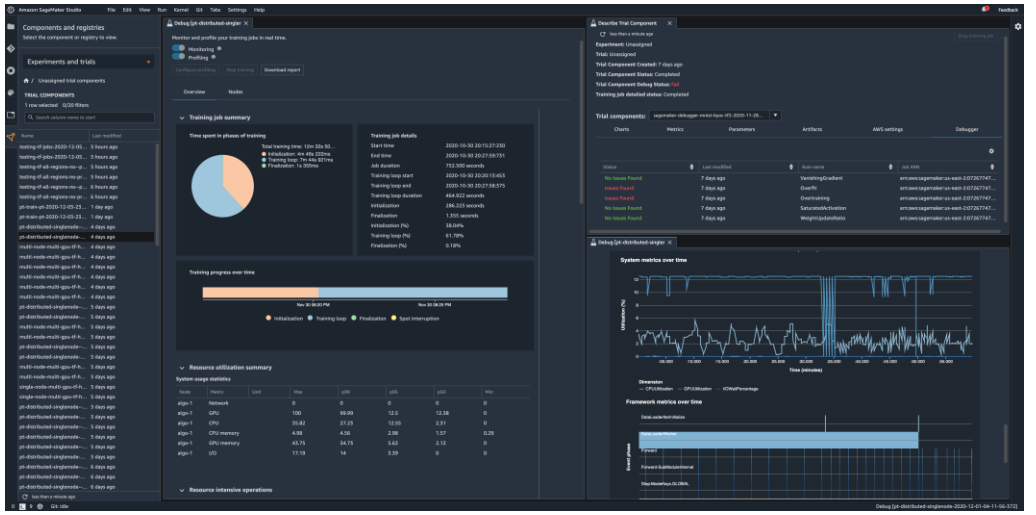
- Saves internal model state at periodical intervals
  - Gradients / tensors over time as a model is trained
  - Define rules for detecting unwanted conditions while training
  - A debug job is run for each rule you configure
  - Logs & fires a CloudWatch event when the rule is hit
- SageMaker Studio Debugger dashboards
- Auto-generated training reports
- Built-in rules:
  - Monitor system bottlenecks
  - Profile model framework operations
  - Debug model parameters

SageMaker Debugger

- Supported Frameworks & Algorithms:
  - Tensorflow
  - PyTorch
  - MXNet
  - XGBoost
  - SageMaker generic estimator (for use with custom training containers)
- Debugger API's available in GitHub
  - Construct hooks & rules for CreateTrainingJob and DescribeTrainingJob API's
  - SMDebug client library lets you register hooks for accessing training data

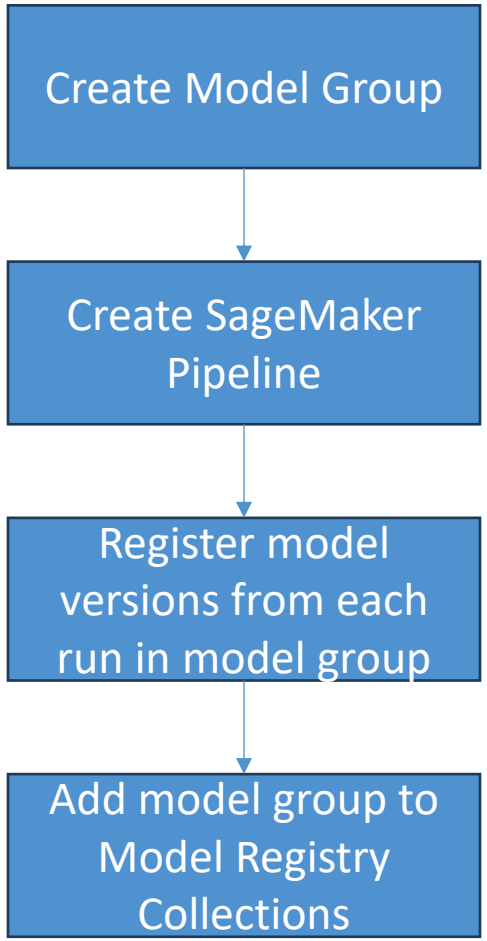
Even Newer SageMaker Debugger Features

- SageMaker Debugger Insights Dashboard
- Debugger ProfilerRule
  - ProfilerReport
  - Hardware system metrics (CPUBottlenck, GPUMemoryIncrease, etc)
  - Framework Metrics (MaxInitializationTime, OverallFrameworkMetrics, StepOutlier)
- Built-in actions to receive notifications or stop training
  - StopTraining(), Email(), or SMS()
  - In response to Debugger Rules
  - Sends notifications via SNS
- Profiling system resource usage and training

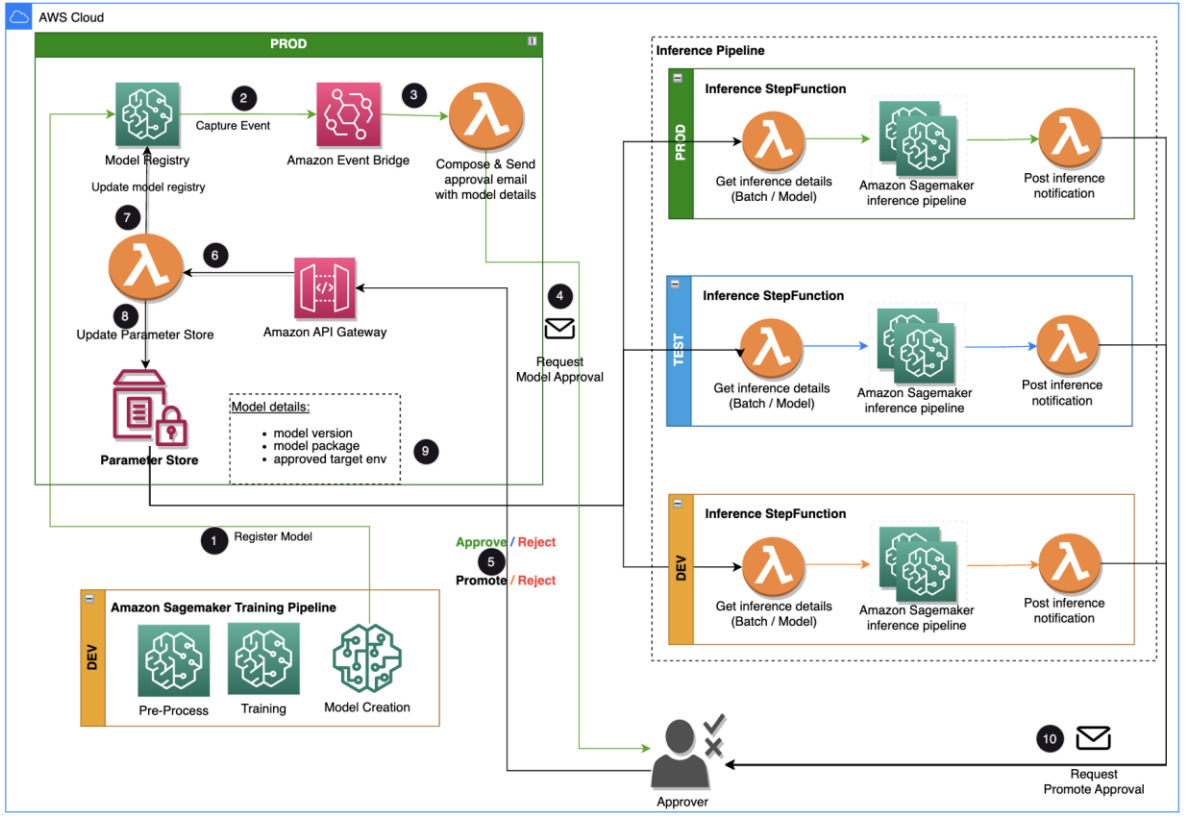


SageMaker Model Registry

- Catalog your models, manage model versions
- Associate metadata with models
- Manage approval status of a model
- Deploy models to production
- Automate deployment with CI/CD
- Share models
- Integrate with SageMaker Model Cards



Example workflow for approving and promoting models with Model Registry



https://aws.amazon.com/blogs/machine-learning/build-an-amazon-sagemaker-model-registry-approval-and-promotion-workflow-with-human-intervention/

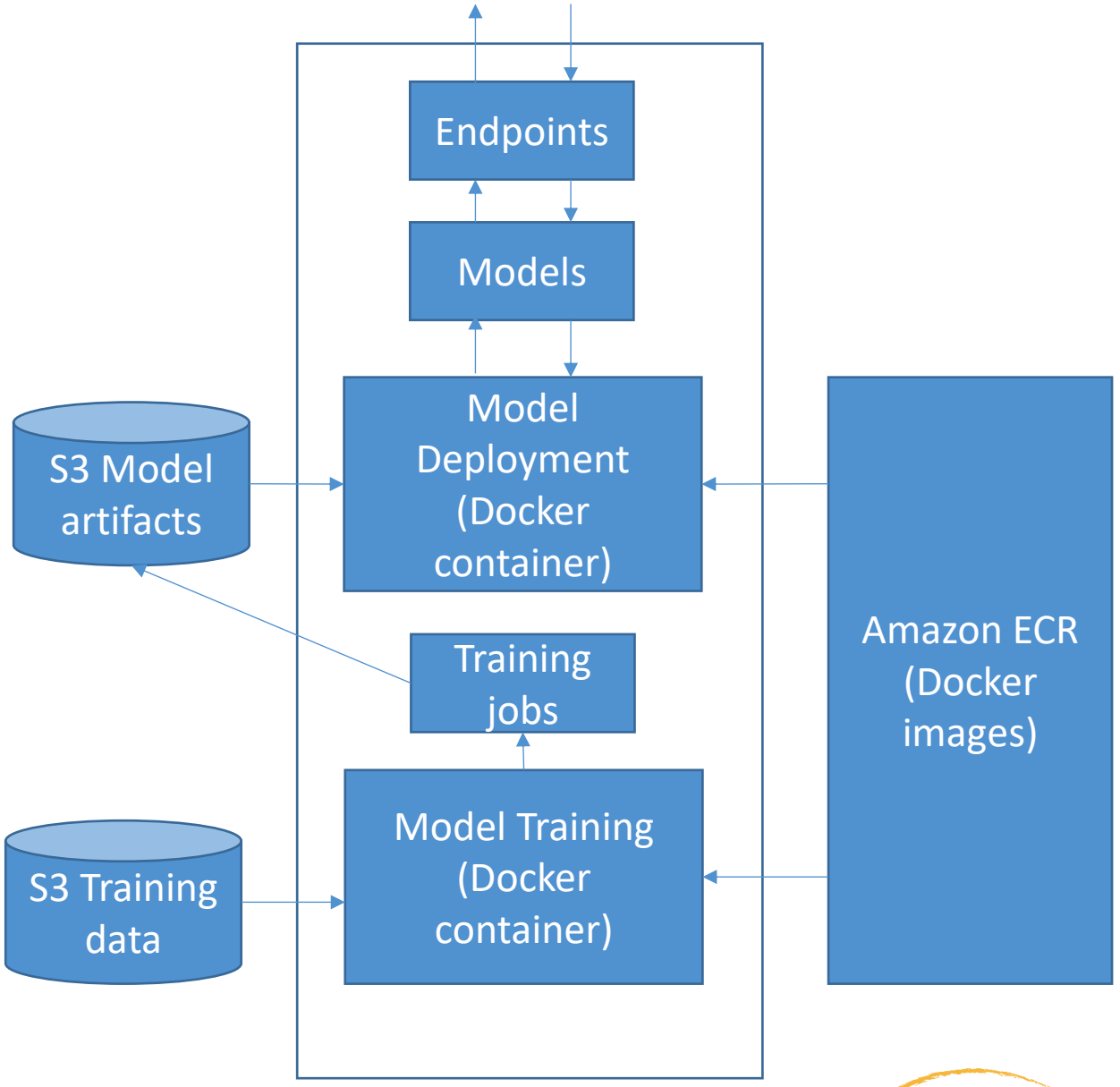
SageMaker and Docker Containers

SageMaker + Docker

- All models in SageMaker are hosted in Docker containers
  - Pre-built deep learning
  - Pre-built scikit-learn and Spark ML
  - Pre-built Tensorflow, MXNet, Chainer, PyTorch
    - Distributed training via Horovod or Parameter Servers
  - Your own training and inference code! Or extend a pre-built image.
- This allows you to use any script or algorithm within SageMaker, regardless of runtime or language
  - Containers are isolated, and contain all dependencies and resources needed to run

Using Docker

- Docker containers are created from *images*
- Images are built from a *Dockerfile*
- Images are saved in a *repository*
  - Amazon Elastic Container Registry



Amazon SageMaker Containers

- Library for making containers compatible with SageMaker
  - `RUN pip install sagemaker-containers` in your Dockerfile

Structure of a training container

```
/opt/ml
├── input
│   ├── config
│   │   ├── hyperparameters.json
│   │   └── resourceConfig.json
│   └── data
│       └── <channel_name>
│           └── <input data>
├── model
│
├── code
│   └── <script files>
│
└── output
└── failure
```

Structure of a Deployment Container

```
/opt/ml
└── model
    └── <model files>
```

Structure of your Docker image

- WORKDIR
  - nginx.conf
  - predictor.py
  - serve/
  - train/
  - wsgi.py

Assembling it all in a Dockerfile

```
FROM tensorflow/tensorflow:2.0.0a0

RUN pip3 install sagemaker-training

# Copies the training code inside the
container
COPY train.py /opt/ml/code/train.py

# Defines train.py as script entrypoint
ENV SAGEMAKER_PROGRAM train.py
```

Environment variables

- SAGEMAKER_PROGRAM
  - Run a script inside /opt/ml/code
- SAGEMAKER_TRAINING_MODULE
- SAGEMAKER_SERVICE_MODULE
- SM_MODEL_DIR
- SM_CHANNELS / SM_CHANNEL_*
- SM_HPS / SM_HP_*
- SM_USER_ARGS
- …and many more

Using your own image

- cd dockerfile
- !docker build -t foo .
- from sagemaker.estimator import Estimator

```
estimator = Estimator(image_name='foo',
                      role='SageMakerRole',
                      train_instance_count=1,
                      train_instance_type='local')

estimator.fit()
```

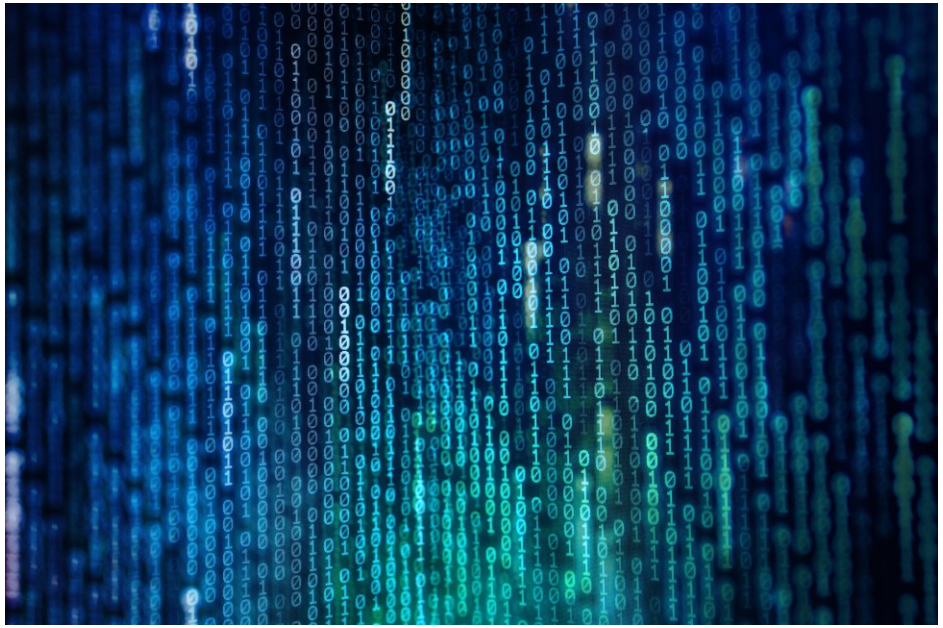
Production Variants

- You can test out multiple models on live traffic using Production Variants
  - Variant Weights tell SageMaker how to distribute traffic among them
  - So, you could roll out a new iteration of your model at say 10% variant weight
  - Once you're confident in its performance, ramp it up to 100%
- This lets you do A/B tests, and to validate performance in real-world settings
  - Offline validation isn't always useful
- Shadow Variants
- Deployment Guardrails

SageMaker on the Edge

SageMaker Neo

- Train once, run anywhere
  - Edge devices
    - ARM, Intel, Nvidia processors
    - Embedded in whatever – your car?
- Optimizes code for specific devices
  - Tensorflow, MXNet, PyTorch, ONNX, XGBoost, DarkNet, Keras
- Consists of a **compiler** and a **runtime**



Neo + AWS IoT Greengrass

- Neo-compiled models can be deployed to an HTTPS endpoint
  - Hosted on C5, M5, M4, P3, or P2 instances
  - Must be same instance type used for compilation
- OR! You can deploy to IoT Greengrass
  - This is how you get the model to an actual edge device
  - Inference at the edge with local data, using model trained in the cloud
  - Uses Lambda inference applications

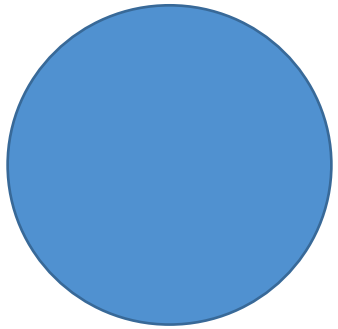
Managing SageMaker Resources

Choosing your instance types

- We covered this under “modeling”, even though it’s an operations concern
- In general, algorithms that rely on deep learning will benefit from GPU instances (P3, g4dn) for training
- Inference is usually less demanding and you can often get away with compute instances there (C5)
- GPU instances can be really pricey

Managed Spot Training

- Can use EC2 Spot instances for training
  - Save up to 90% over on-demand instances
- Spot instances can be interrupted!
  - Use checkpoints to S3 so training can resume
- Can increase training time as you need to wait for spot instances to become available



Automatic Scaling

- You set up a scaling policy to define target metrics, min/max capacity, cooldown periods
- Works with CloudWatch
- Dynamically adjusts number of instances for a production variant
- Load test your configuration before using it!

Deploying Models for Inference

- SageMaker JumpStart
  - Deploying pre-trained models to pre-configured endpoints
- ModelBuilder from the SageMaker Python SDK
  - Configure deployment settings from code
- AWS CloudFormation
  - For advanced users who need consistent and repeatable deployments
  - AWS::SageMaker::Model resources creates a model to host at an endpoint

Deploying Models for Inference

- Real-time Inference
  - For interactive workloads with low latency requirements
- Amazon SageMaker Serverless Inference
  - No management of infrastructure
  - Ideal if workload has idle periods and uneven traffic over time, and can tolerate cold starts
- Asynchronous Inference
  - Queues requests and processes them asynchronously
  - Example: large payload sizes (up to 1GB) with long processing times, but near-real-time latency requirements
- Autoscaling
  - Dynamically adjust compute resources for endpoints based on traffic
- SageMaker Neo
  - Optimizes models for AWS Inferentia chips (for example)

Serverless Inference

- Introduced for 2022
- Specify your container, memory requirement, concurrency requirements
- Underlying capacity is automatically provisioned and scaled
- Good for infrequent or unpredictable traffic; will scale down to zero when there are no requests.
- Charged based on usage
- Monitor via CloudWatch
  - ModelSetupTime, Invocations, MemoryUtilization

Amazon SageMaker Serverless Inference

Fully managed serverless endpoints for machine learning inference with pay-per-use pricing

Amazon SageMaker Inference Recommender

- Recommends best instance type & configuration for your models
- Automates load testing model tuning
- Deploys to optimal inference endpoint
- How it works:
  - Register your model to the model registry
  - Benchmark different endpoint configurations
  - Collect & visualize metrics to decide on instance types
  - Existing models from zoos may have benchmarks already
- Instance Recommendations
  - Runs load tests on recommended instance types
  - Takes about 45 minutes
- Endpoint Recommendations
  - Custom load test
  - You specify instances, traffic patterns, latency requirements, throughput requirements
  - Takes about 2 hours

```
'InferenceRecommendations': [{
        'Metrics': {
          'CostPerHour': 0.20399999618530273,
          'CostPerInference': 5.246913588052848e-06,
          'MaximumInvocations': 648,
          'ModelLatency': 263596
          },
        'EndpointConfiguration': {
          'EndpointName': 'endpoint-name',
          'VariantName': 'variant-name',
          'InstanceType': 'ml.c5.xlarge',
          'InitialInstanceCount': 1
          },
        'ModelConfiguration': {
          'Compiled': False,
          'EnvironmentParameters': []
          }
    },
```

SageMaker and Availability Zones

- SageMaker automatically attempts to distribute instances across availability zones
- But you need more than one instance for this to work!
- Deploy multiple instances for each production endpoint
- Configure VPC's with at least two subnets, each in a different AZ

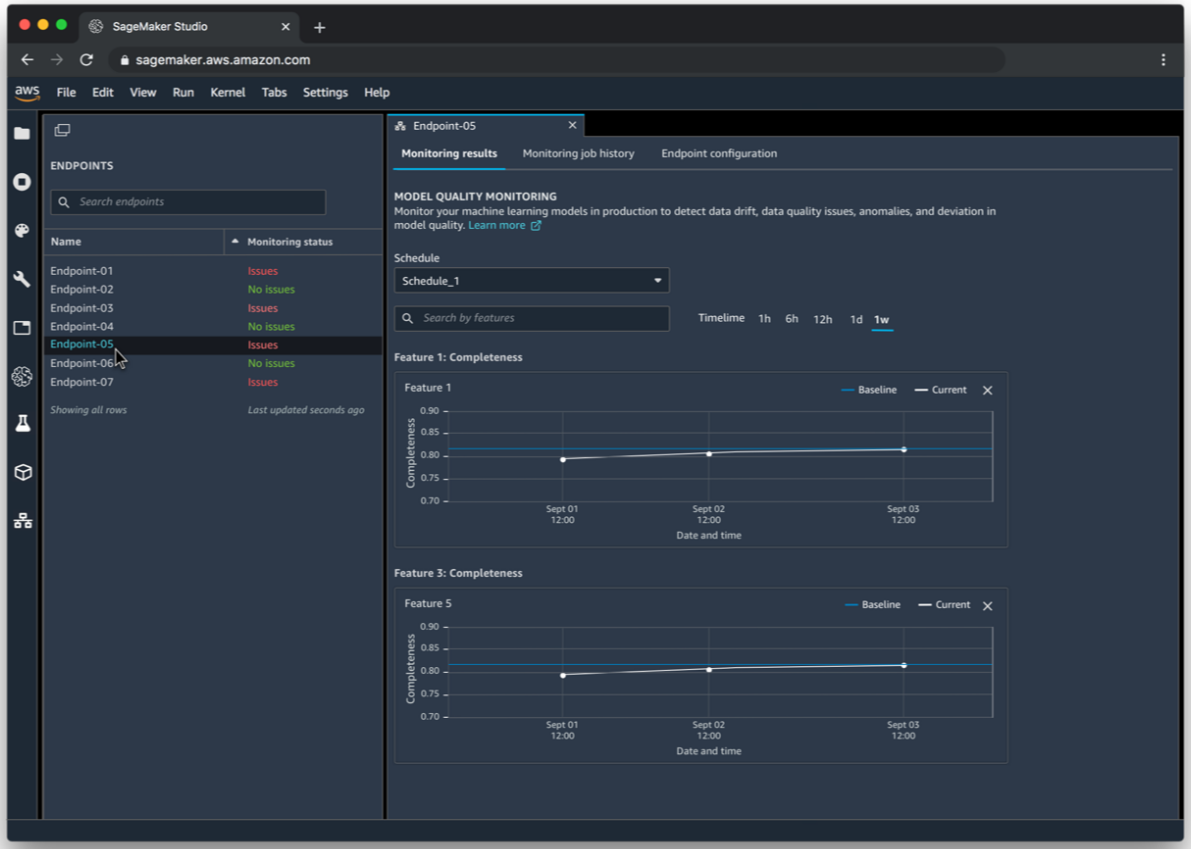
Inference Pipelines

Inference Pipelines

- Linear sequence of 2-15 containers
- Any combination of pre-trained built-in algorithms or your own algorithms in Docker containers
- Combine pre-processing, predictions, post-processing
- Spark ML and scikit-learn containers OK
  - Spark ML can be run with Glue or EMR
  - Serialized into MLeap format
- Can handle both real-time inference and batch transforms

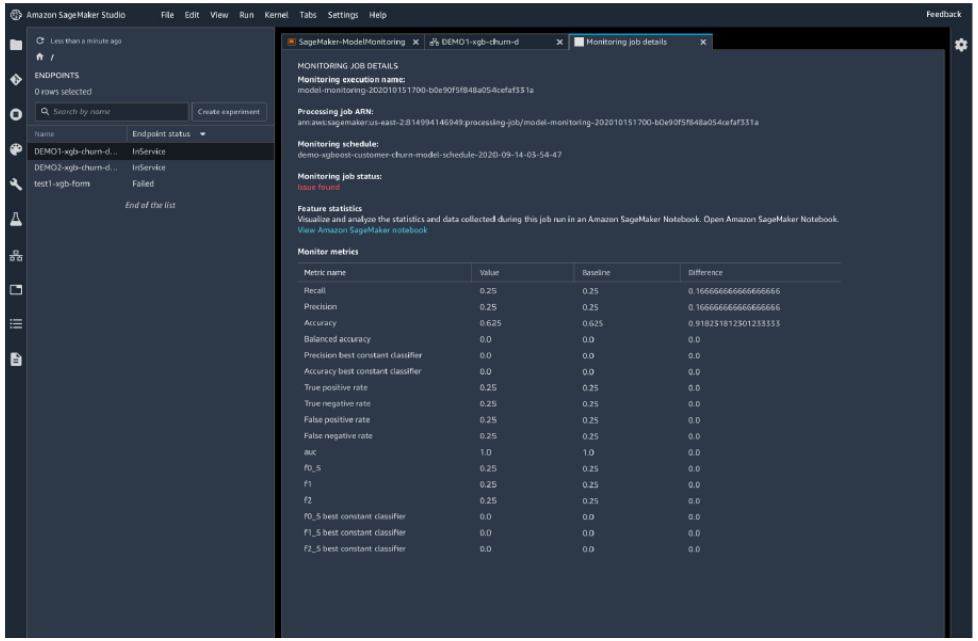
SageMaker Model Monitor

- Get alerts on quality deviations on your deployed models (via CloudWatch)
- Visualize data drift
  - Example: loan model starts giving people more credit due to drifting or missing input features
- Detect anomalies & outliers
- Detect new features
- No code needed



SageMaker Model Monitor + Clarify

- Integrates with SageMaker Clarify
  - SageMaker Clarify detects potential bias
  - i.e., imbalances across different groups / ages / income brackets
  - With ModelMonitor, you can monitor for bias and be alerted to new potential bias via CloudWatch
  - SageMaker Clarify also helps explain model behavior
    - Understand which features contribute the most to your predictions

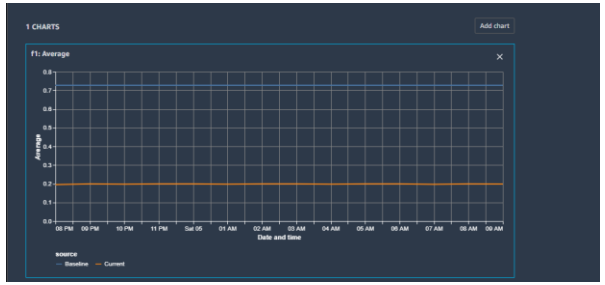


Pre-training Bias Metrics in Clarify

- Class Imbalance (CI)
  - One facet (demographic group) has fewer training values than another
- Difference in Proportions of Labels (DPL)
  - Imbalance of positive outcomes between facet values
- Kullback-Leibler Divergence (KL), Jensen-Shannon Divergence(JS)
  - How much outcome distributions of facets diverge
- Lp-norm (LP)
  - P-norm difference between distributions of outcomes from facets
- Total Variation Distance (TVD)
  - L1-norm difference between distributions of outcomes from facets
- Kolmogorov-Smirnov (KS)
  - Maximum divergence between outcomes in distributions from facets
- Conditional Demographic Disparity (CDD)
  - Disparity of outcomes between facets as a whole, and by subgroups

SageMaker Model Monitor

- Data is stored in S3 and secured
- Monitoring jobs are scheduled via a Monitoring Schedule
- Metrics are emitted to CloudWatch
  - CloudWatch notifications can be used to trigger alarms
  - You'd then take corrective action (retrain the model, audit the data)
- Integrates with Tensorboard, QuickSight, Tableau
  - Or just visualize within SageMaker Studio



SageMaker Model Monitor

- Monitoring Types:
  - Drift in data quality
    - Relative to a baseline you create
    - “Quality” is just statistical properties of the features
  - Drift in model quality (accuracy, etc)
    - Works the same way with a model quality baseline
    - Can integrate with Ground Truth labels
  - Bias drift
  - Feature attribution drift
    - Based on Normalized Discounted Cumulative Gain (NDCG) score
    - This compares feature ranking of training vs. live data

Model Monitor Data Capture

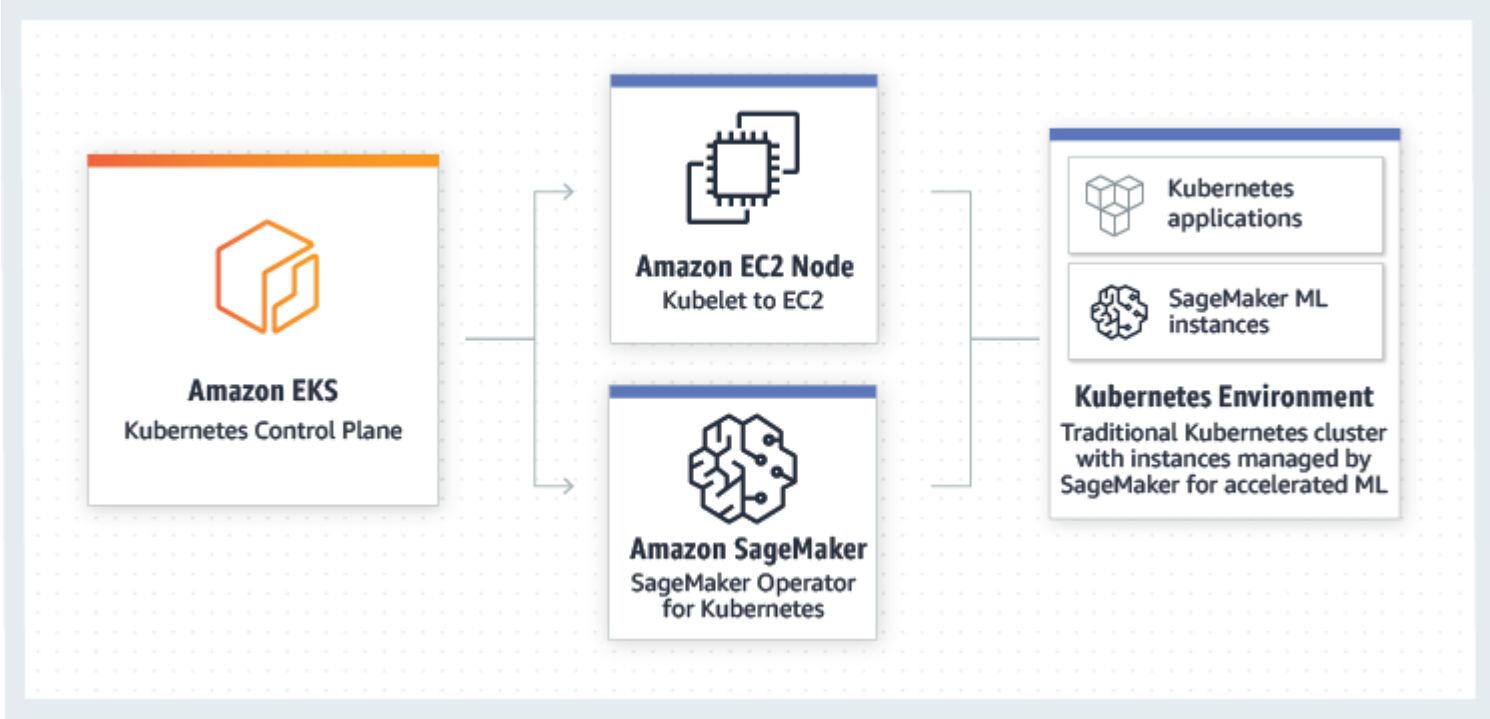
- Logs inputs to your endpoint and inference outputs
  - Data delivered to S3 as JSON
- This can be used for
  - Further training
  - Debugging
  - Monitoring
- Automatically compares data metrics to your baseline
- Supported for both real-time and batch model monitor modes
- Supported for Python (Boto) and SageMaker Python SDK
- Inference data may be encrypted

MLOps with SageMaker

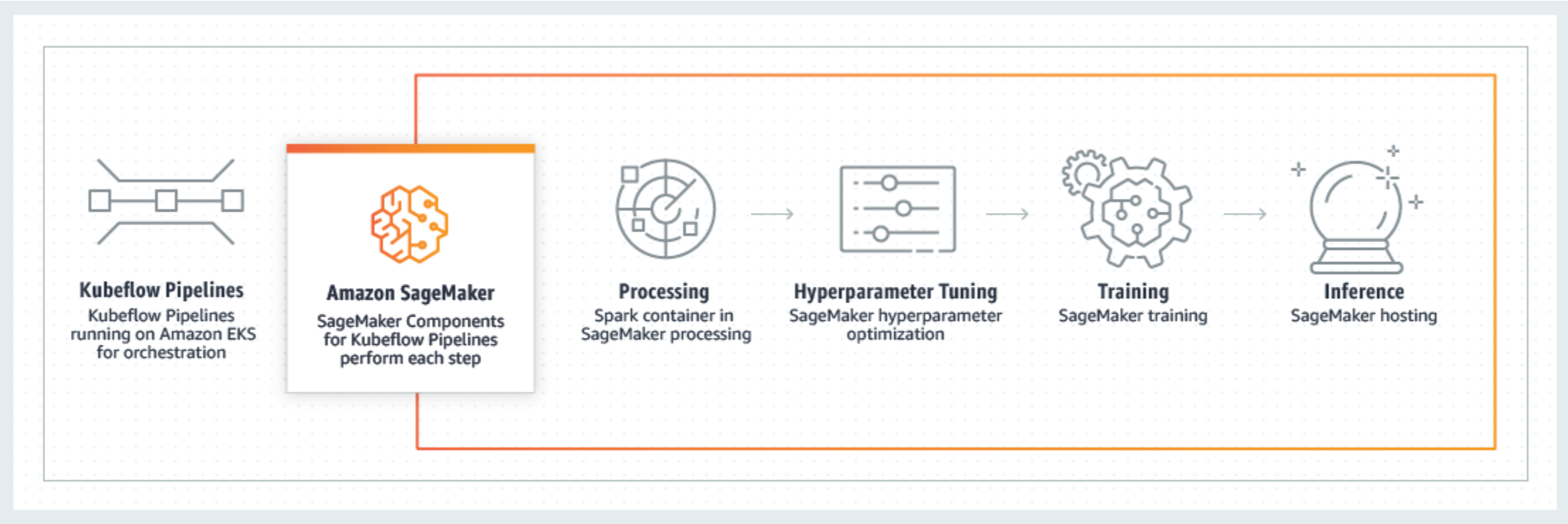
MLOps with SageMaker and Kubernetes

- Integrates SageMaker with Kubernetes-based ML infrastructure
- Amazon SageMaker Operators for Kubernetes
- Components for Kubeflow Pipelines
- Enables hybrid ML workflows (on-prem + cloud)
- Enables integration of existing ML platforms built on Kubernetes / Kubeflow

SageMaker Operators for Kubernetes

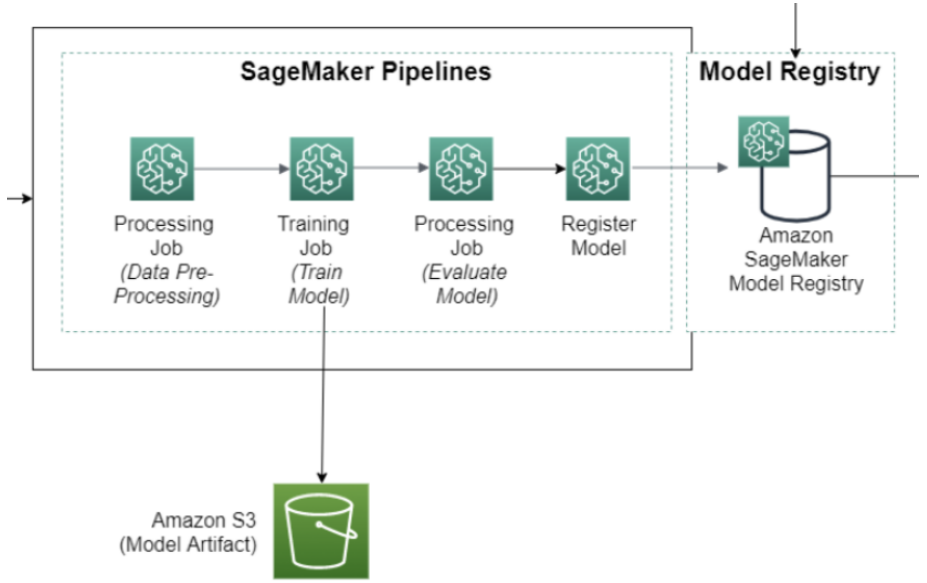


SageMaker Components for Kubeflow Pipelines

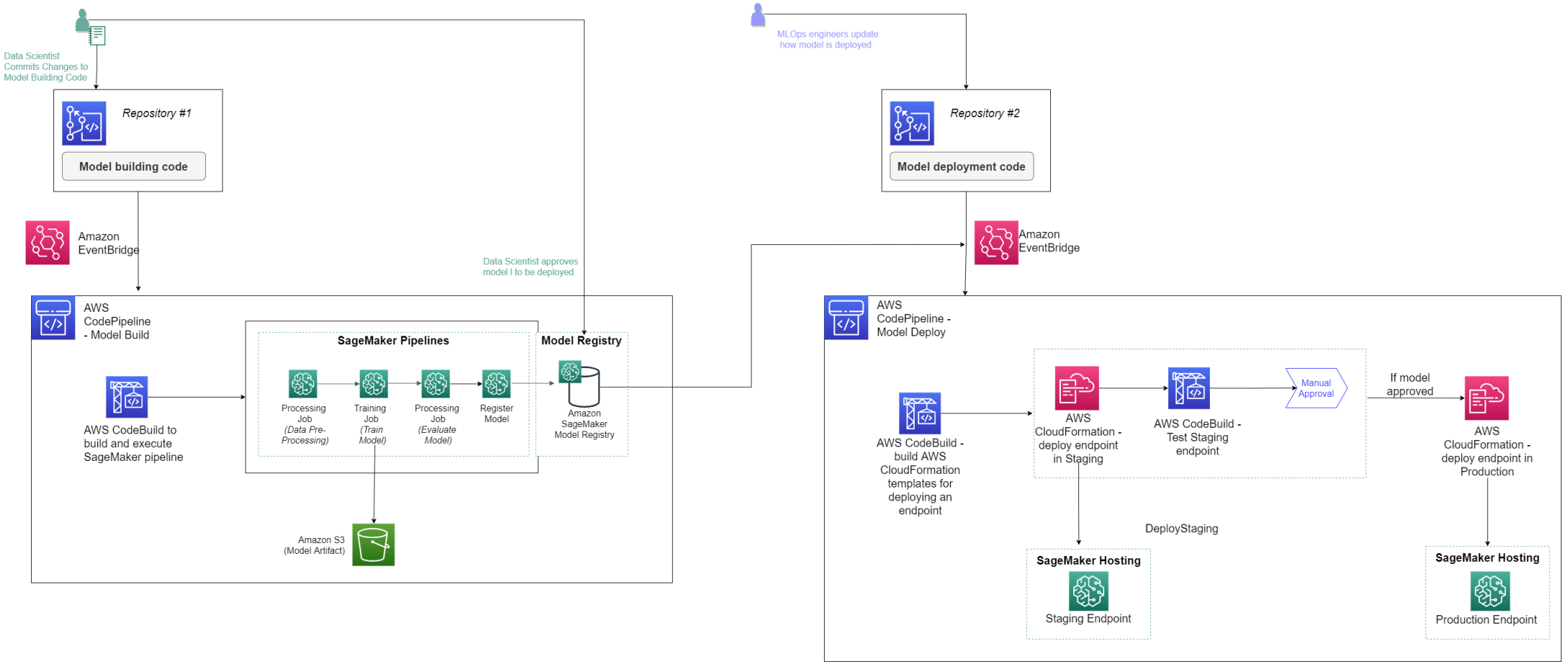


SageMaker Projects

- SageMaker Studio's native MLOps solution with CI/CD
  - Build images
  - Prep data, feature engineering
  - Train models
  - Evaluate models
  - Deploy models
  - Monitor & update models
- Uses code repositories for building & deploying ML solutions
- Uses SageMaker Pipelines defining steps

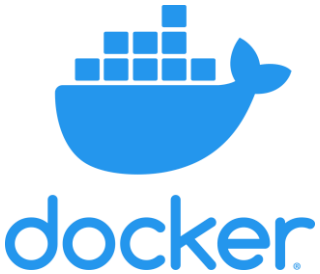


SageMaker Projects



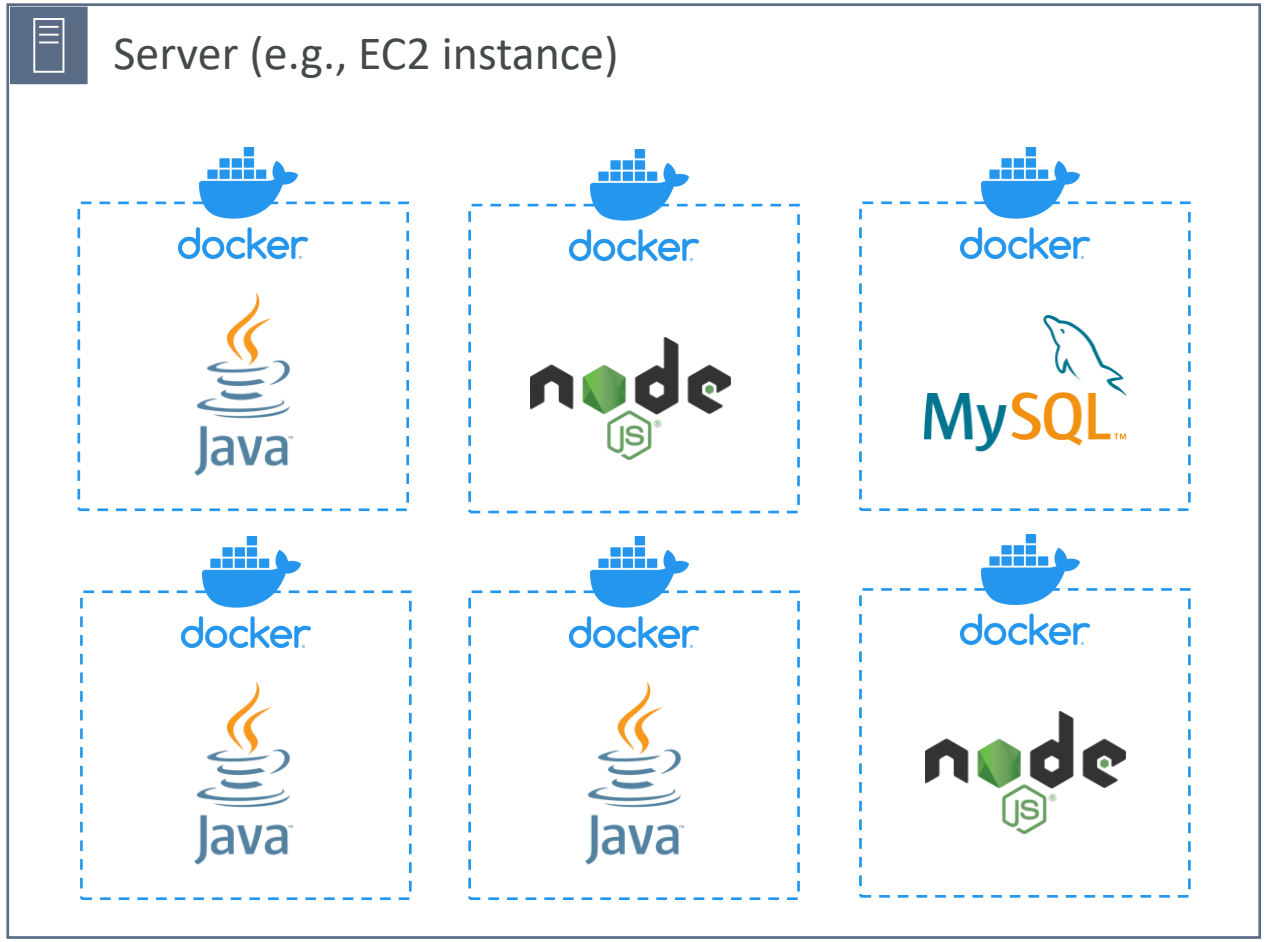
Containers on AWS

What is Docker?



- Docker is a software development platform to deploy apps
- Apps are packaged in **containers** that can be run on any OS
- Apps run the same, regardless of where they're run
  - Any machine
  - No compatibility issues
  - Predictable behavior
  - Less work
  - Easier to maintain and deploy
  - Works with any language, any OS, any technology
- Use cases: microservices architecture, lift-and-shift apps from on-premises to the AWS cloud, …

Docker on an OS



Where are Docker images stored?

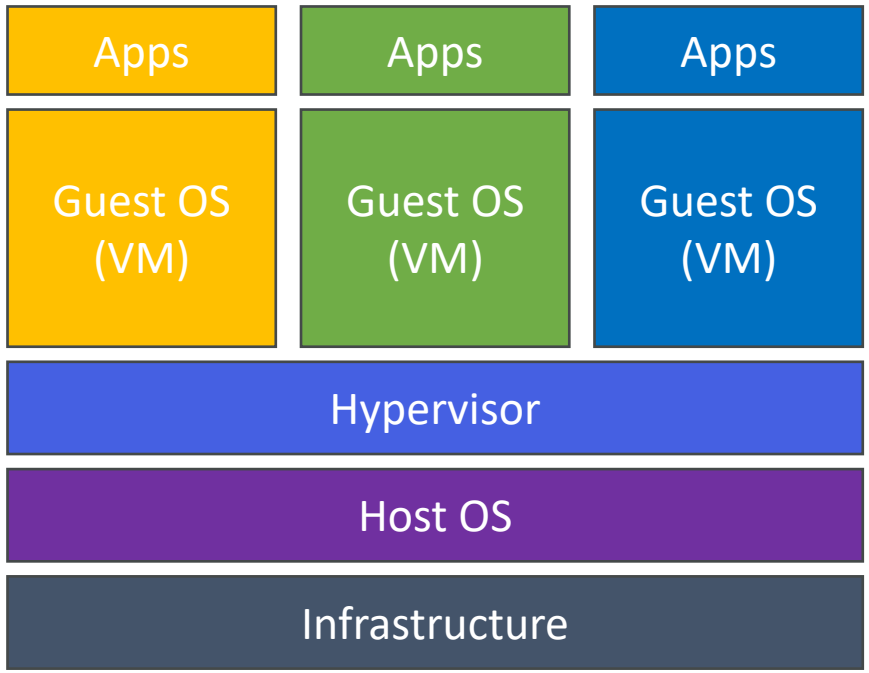
- Docker images are stored in Docker Repositories

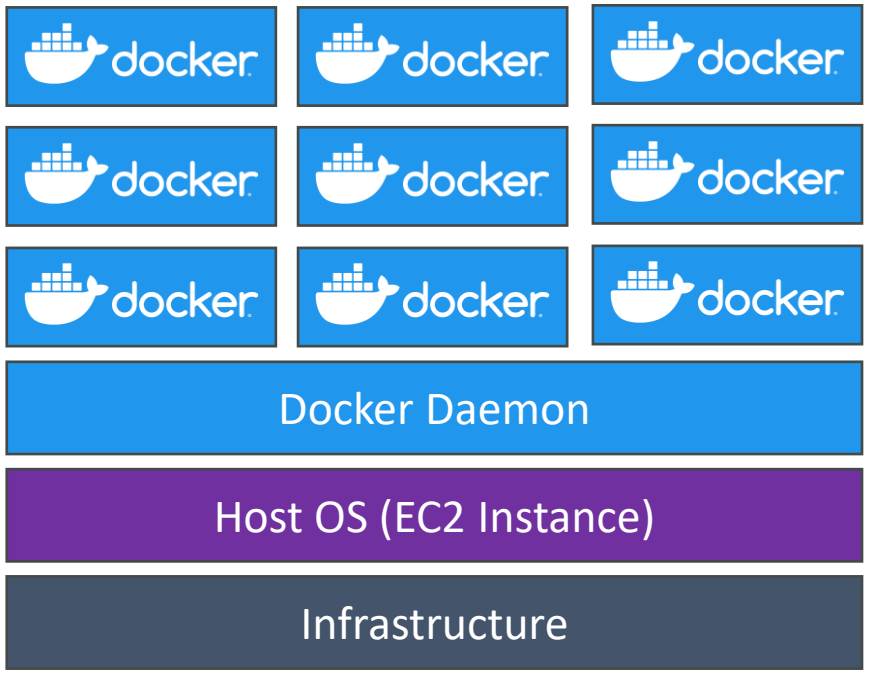
- **Docker Hub (https://hub.docker.com)**
  - **Public** repository
  - Find base images for many technologies or OS (e.g., Ubuntu, MySQL, …)

- **Amazon ECR (Amazon Elastic Container Registry)**
  - **Private** repository
  - **Public** repository (**Amazon ECR Public Gallery** https://gallery.ecr.aws)

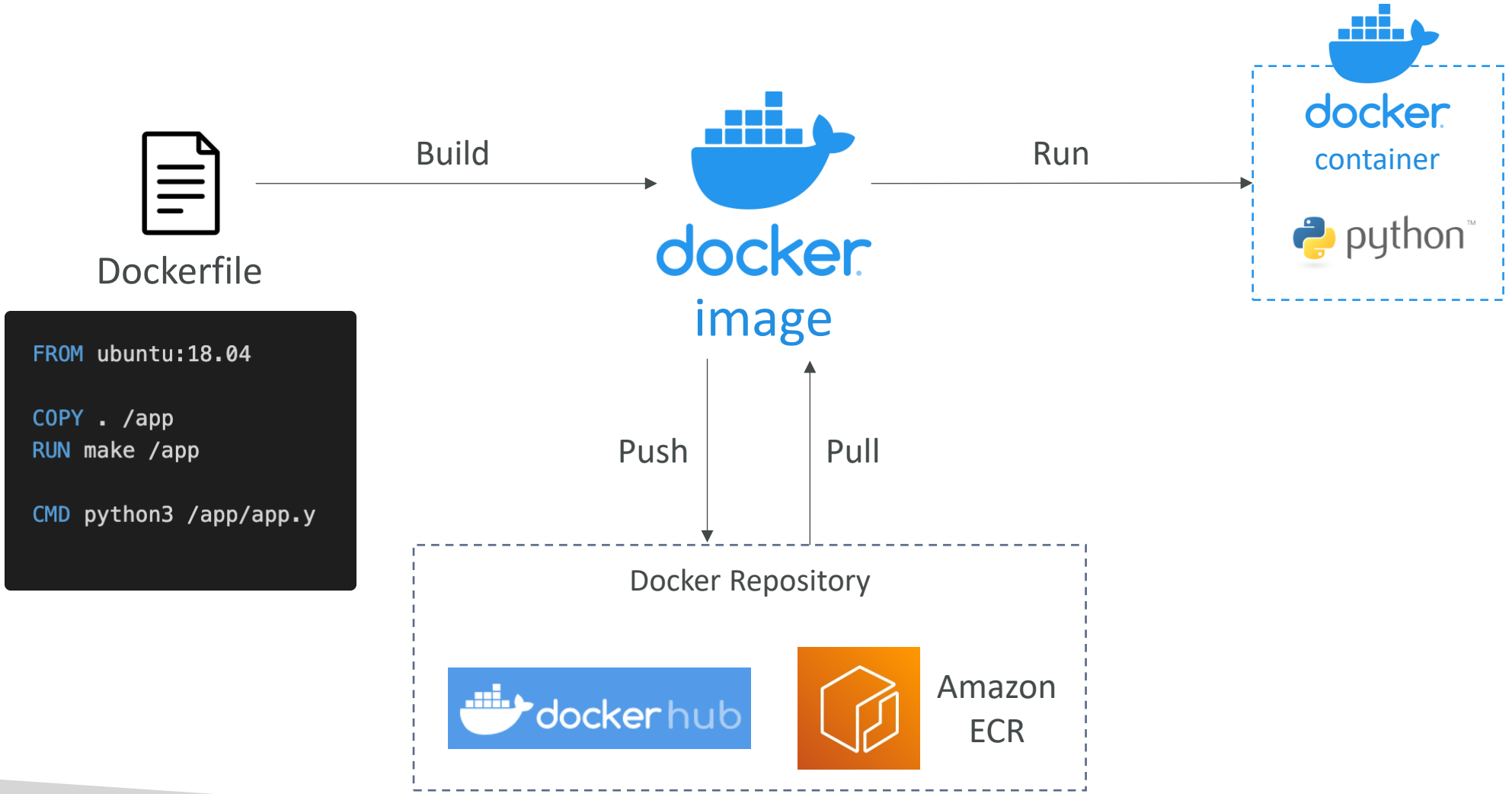
Docker vs. Virtual Machines

- Docker is ”sort of” a virtualization technology, but not exactly
- Resources are shared with the host => many containers on one server





Getting Started with Docker



Docker Containers Management on AWS

- **Amazon Elastic Container Service (Amazon ECS**
  - Amazon's own container platform

Amazon ECS

- **Amazon Elastic Kubernetes Service (Amazon EK**
  - Amazon's managed Kubernetes (open source)

Amazon EKS

- **AWS Fargate**
  - Amazon's own Serverless container platform
  - Works with ECS and with EKS

AWS Fargate

- **Amazon ECR:**
  - Store container images

Amazon ECR

DataCumulus

Sundog Education